DM587

Scientific Programming

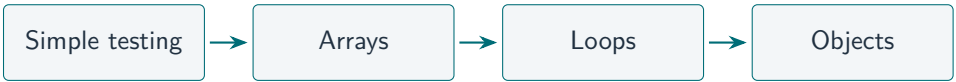
Lecture 02: Arrays, loops and objects

Sai Ganesh Nagarajan

University of Southern Denmark

21 October

Today



Running a file

```
# squares.py
def square(x):
    return x * x

if __name__ == "__main__":
    print(square(3))
```

Run it directly

`python3 squares.py`
Python sets `__name__` to `"__main__"`.
The condition is true, so the test prints 9.

Import it in another Python file

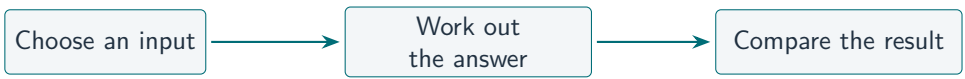
`import squares`
Python sets `__name__` inside `squares.py` to `"squares"`. The condition is false, so the test prints nothing.

The function is still available:
`print(squares.square(3))` prints 9.

The if block runs the example only when this file is run directly.

Checking results

```
if __name__ == "__main__":
    print("square(0):", square(0), "expected:", 0)
    print("square(-3):", square(-3), "expected:", 9)
```



print accepts several values, separated by commas.
It places spaces between them and ends with a new line.

Part 1

Arrays

Select, reshape and combine data

Section reading

ACME Lab 4: Intro to NumPy

Shape

Temperatures in °C: four readings a day, over three days.

```
T = np.array([[12, 16, 19, 14],
              [11, 15, 18, 13],
              [10, 14, 17, 12]])
```

	Morning	Midday	Afternoon	Evening
Day 1	12	16	19	14
Day 2	11	15	18	13
Day 3	10	14	17	12

`T[1, 2]` = 18: afternoon on Day 2.

`T.shape` = (3, 4): rows are days; columns are times.

Slicing

```
B = T[:2, 1:3]
```

	Morning	Midday	Afternoon	Evening
Day 1	12	16	19	14
Day 2	11	15	18	13
Day 3	10	14	17	12



16	19
15	18

`B.shape` $= (2, 2)$

Midday and afternoon, on the first two days.

The start is included; the stop is excluded.

A basic slice is a view: editing B also edits T.

Use `B = T[:2, 1:3].copy()` for independent data.

Dimensions

```
T[1, :]      # Day 2: a 1D array
T[1:2, :]    # Day 2: keep the day axis
```

`T[1, :]`: shape (4,)

11	15	18	13

`T[1:2, :]`: shape (1, 4)

11	15	18	13

Same readings; different numbers of dimensions.

An integer selects one row as a 1D array;
a slice keeps the result as a 2D table.

`T[1, 2]` removes both axes and returns a scalar.

Your turn

	Morning	Midday	Afternoon	Evening
Day 1	12	16	19	14
Day 2	11	15	18	13
Day 3	10	14	17	12

1. Afternoon on all three days?
2. All readings from the last two days?
3. Morning and afternoon on the first two days?

Write a slice and predict its values and shape.

Answers

Slice	Values and shape
`T[:, 2]`	`[19, 18, 17]`, shape (3,)
`T[1:, :]`	`[[11, 15, 18, 13],` `[10, 14, 17, 12]]`, shape (2, 4)
`T[:2, ::2]`	`[[12, 19], [11, 18]]`, shape (2, 2)

`::2` takes every second column, starting at column 0.

Alternative: `T[:2, [0, 2]]` selects columns 0 and 2 explicitly.

List-based indexing can select arbitrary columns and returns a copy; basic slicing gives a view.

Reshaping

A device saves a flat list of readings. We want a table: days as rows, times as columns.

Saved day by day

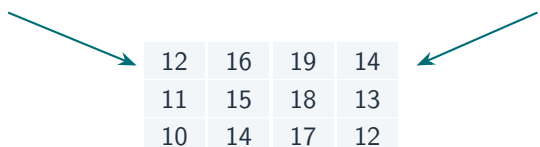
```
day_order = np.array([
    12, 16, 19, 14, 11, 15,
    18, 13, 10, 14, 17, 12])
day_order.reshape(3, 4, order="C")
```

First four entries: all readings on Day 1.

Saved time by time

```
time_order = np.array([
    12, 11, 10, 16, 15, 14,
    19, 18, 17, 14, 13, 12])
time_order.reshape(3, 4, order="F")
```

First three entries: mornings on all days.



12	16	19	14
11	15	18	13
10	14	17	12

Both reconstruct the same table when the order matches how readings were saved.

Stacking

Extend the same table: add a day, or add a night-time reading.

```
new_day = np.array([[9, 13, 16, 11]])
night = np.array([[8], [7], [6]])
np.vstack((T, new_day))    # (4, 4)
np.hstack((T, night))      # (3, 5)
```

Another day: append a row

12	16	19	14
11	15	18	13
10	14	17	12
9	13	16	11

Another time: append a column

12	16	19	14	8
11	15	18	13	7
10	14	17	12	6

Vertical: columns must match. Horizontal: rows must match.

The same idea joins compatible matrix blocks.

Predict the shape

1D arrays: shape (2,)

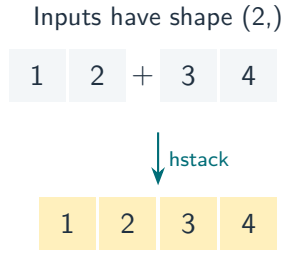
```
a = np.array([1, 2])
b = np.array([3, 4])
np.hstack((a, b))
```

2D columns: shape (2, 1)

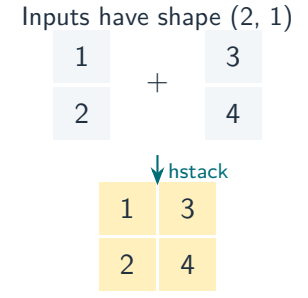
```
a = np.array([[1], [2]])
b = np.array([[3], [4]])
np.hstack((a, b))
```

- What values and shape does each result have?
- Does horizontal stacking produce the same arrangement?

Shapes



`[1, 2, 3, 4]`
Shape (4,): join end to end.

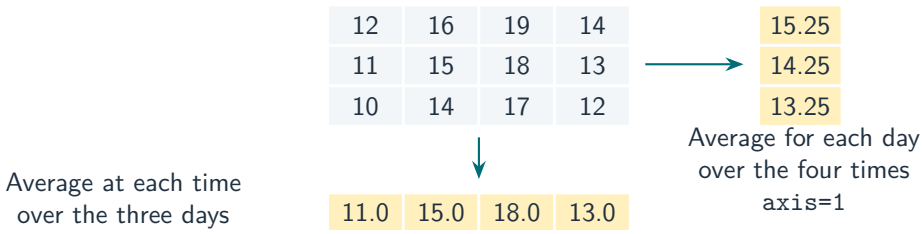


`[[1, 3], [2, 4]]`
Shape (2, 2): join columns.

(2,) has one axis; (2, 1) has two. The extra axis changes the result.

Axes

```
T.mean(axis=0)   # Average at each time of day
T.mean(axis=1)   # Average for each day
```



Broadcasting

Suppose the instrument needs a different correction at each time of day.

```
correction = np.array([0, 1, -1, 0])
corrected = T + correction
```

Original readings

12	16	19	14
11	15	18	13
10	14	17	12

+

0	1	-1	0

Apply this correction to every day.

Corrected readings

12	17	18	14
11	16	17	13
10	15	16	12

A length-4 array matches the four columns; no Python loop is needed.

Part 2

Loops and comprehensions

Visit items; transform them; keep the ones you need

Section reading

ACME Lab 1: loops and list comprehension

Loop basics

for: visit the values

```
readings = [12, 16, 19]
for value in readings:
    print(value)
```

Prints 12, 16, 19 on separate lines.

range: integer steps

```
for i in range(3):
    print(i)
```

Visits 0, 1, 2; the stop is excluded.

enumerate: index and value

```
for i, value in enumerate(readings):
    print(i, value)
```

Prints pairs: 0 12, then 1 16, then 2 19.

np.arange: a numerical array

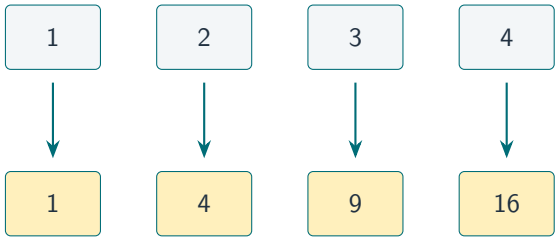
```
times = np.arange(0, 1, 0.25)
# [0.0, 0.25, 0.5, 0.75]
```

Start, stop, step; the stop is excluded.

Loop over values directly; use enumerate when you also need their indices.

Iteration

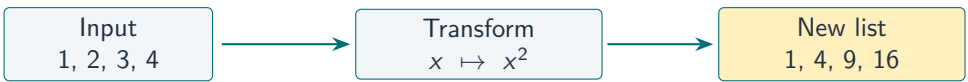
```
for x in [1, 2, 3, 4]:
    print(x * x)
```



Visit one item, do the operation, move to the next.

Comprehensions

```
squares = [x * x for x in [1, 2, 3, 4]]
```

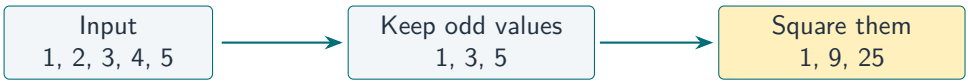


`[expression for item in iterable]`

Read the input first, then the expression that transforms each item.

Filtering

```
odd_squares = [x * x for x in range(1, 6) if x % 2 == 1]
```



The trailing `if` filters items before the expression is evaluated.
`range(1, 6)` supplies 1 through 5.

Predict the lists

```
values = [-2, -1, 0, 1, 2, 3]
```

1. Two conditions

```
[x for x in values if x > 0 and x % 2 == 0]
```

2. Filter, then transform

```
[x * x for x in values if x < 0]
```

3. Choose a value for every input

```
[x if x > 0 else 0 for x in values]
```

Predict each list. Which expressions keep every input position?

Answers

1. `[2]`

   Keep values that are both positive and even.

2. `[4, 1]`

   Keep the negative inputs, then square them. Their original order is preserved.

3. `[0, 0, 0, 1, 2, 3]`

   Keep all six positions. Replace each non-positive value with zero.

Filter or choose?

`[expression for x in values if condition]` filters inputs.

`[a if condition else b for x in values]` chooses an output for each input.

Pairs

```
pairs = [(i, j) for i in [1, 2] for j in [3, 4, 5]]
```

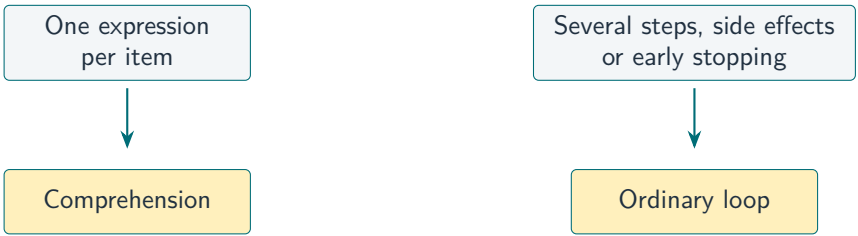
Result: one list containing six tuples

```
[(1, 3), (1, 4), (1, 5), (2, 3), (2, 4), (2, 5)]
```

- For `i = 1`, visit `j = 3, 4, 5`: append the first three tuples.
- Then for `i = 2`, visit `j = 3, 4, 5`: append the next three.

The first for is the outer loop. Each item in the result is a tuple.

Classic loop vs comprehension

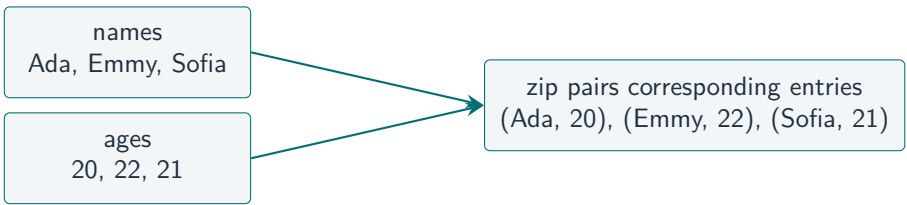


Choose the form that makes the calculation easy to follow.
A comprehension builds a new collection; a loop can do more than build one.

Paired data

`names` and `ages` are two separate lists; matching positions describe the same person.

```
names = ["Ada", "Emmy", "Sofia"]
ages = [20, 22, 21]
for name, age in zip(names, ages):
    print(name, age)
```



zip stops when the shorter list runs out; it does not make every possible pair.

Recursion

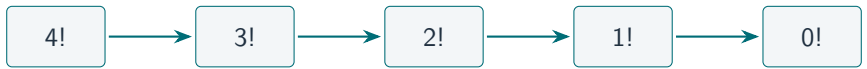
$n! = n(n-1)!$, with $0! = 1$. Assume n is a non-negative integer.

Recursive function

```
def factorial(n):
    if n == 0:
        return 1
    return n * factorial(n - 1)
```

For loop

```
def factorial_loop(n):
    result = 1
    for k in range(1, n + 1):
        result *= k
    return result
```



One chain of calls; no repeated subproblem.
Both make n multiplications; recursion also keeps n pending calls.

A loop is usually faster here and avoids Python's recursion-depth limit.

Memoization

First call: `factorial_memo(4)` with only $0! = 1$ cached.



return values: multiply on the way back and save each result

Cache afterwards: `{0: 1, 1: 1, 2: 2, 3: 6, 4: 24}`

Next call: `factorial_memo(5)`



Return $5 \times 24 = 120$.
No calls to $3!, 2!, 1!, 0!$.

Caching helps across repeated calls; it saves no work in the first chain.

Part 3

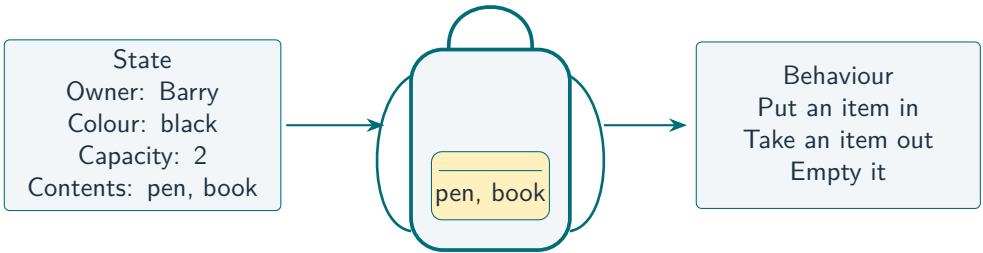
Objects

State, behaviour and reusable designs

Section reading

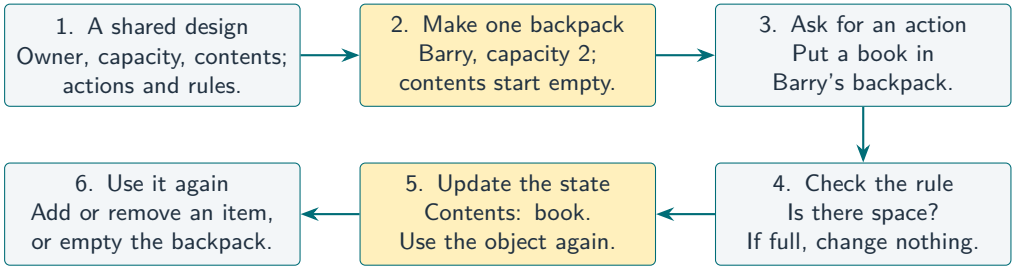
ACME Lab 3: Object-oriented Programming

A backpack



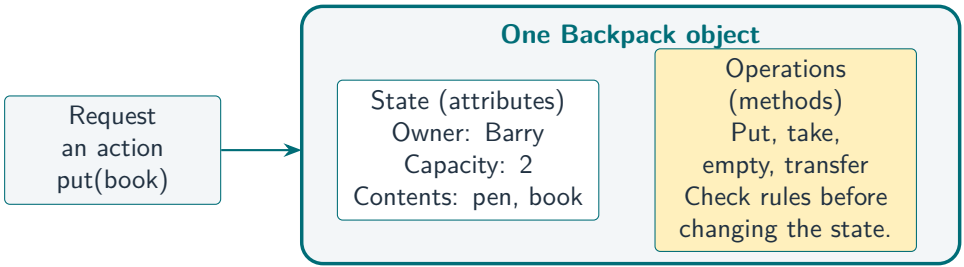
An object keeps related data and the actions that use that data together.

A backpack in use



Each backpack has its own state. Actions follow the same shared rules.

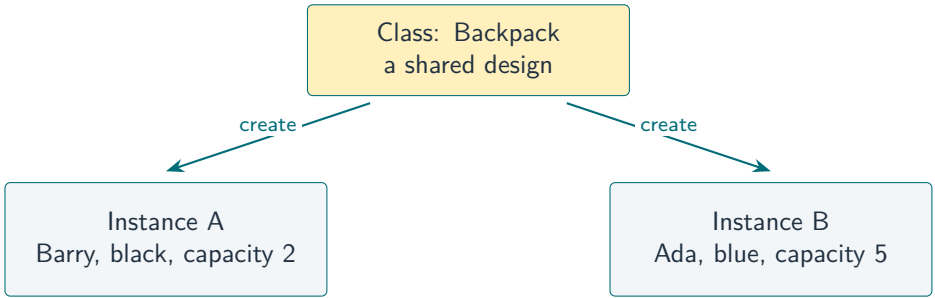
Encapsulation



Encapsulation motivates classes: bundle state with the operations that use it.

For state alone, a dictionary may suffice.

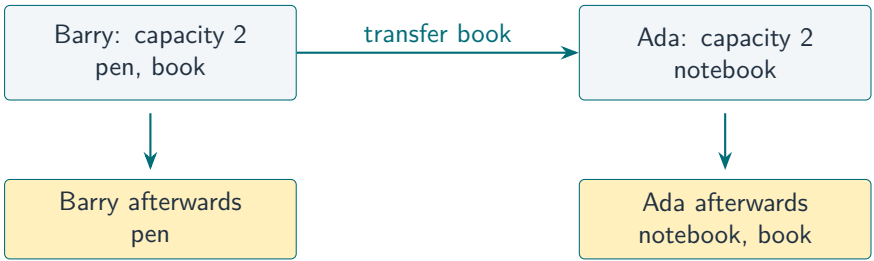
Class and instance



Same kind of object; different state.

Interactions

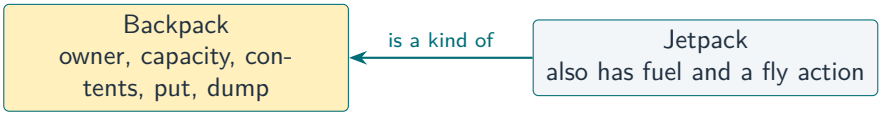
Move the book from Barry's backpack to Ada's backpack.



First check that Barry has the item and Ada has room.
If either check fails, leave both backpacks unchanged.

One operation coordinates the state of two individual objects.

Inheritance



Inherited unchanged: put(item)
Uses Backpack's method.
Checks the same capacity rule before adding an item.

Overridden: dump()
Backpack: empty the contents.
Jetpack: empty the contents and set fuel to zero.

The overridden dump can reuse Backpack's emptying action, then clear the fuel.

Bank accounts

Alice and Bob each have a bank account. Design a class that could represent both.



1. What information should each account store? Give the two instances different starting values.
2. Which methods act on one account? Which could involve another account?
3. What rules should those methods check or preserve?
4. Can you suggest specialised account types? What would they inherit, add or change?

Explore in words and diagrams. There is more than one sensible design.

Possible choices

One account
Deposit money
Withdraw money
Show the balance
List transactions

Between accounts
Transfer money
Compare balances
Calculate a combined balance

Rules
Positive amounts
Sufficient funds
Transfers preserve the total
Active accounts

Possible subclasses
Savings: interest
Current: overdraft
Student: fewer fees
Business: transaction fees

Possible state: owner, account number, balance; add history or status if your methods need them.

From design to Python

Idea	Python spelling
Shared design	`class Backpack:`
An individual backpack	`pack = Backpack("Barry", "black")`
Its starting state	`def __init__(self, ...):`
An attribute	`self.contents`
An operation	`def put(self, item):`
Ask that object to act	`pack.put("book")`

self is the object receiving the call: here, pack.

Calling `Backpack(...)` creates an instance and initialises its own state.

Constructor

```
class Backpack:
    def __init__(self, name, color, max_size=2):
        self.name = name
        self.color = color
        self.max_size = max_size
        self.contents = []
```

- `self.name`: an attribute belonging to this instance.
- `max_size=2`: an ordinary default argument.
- A new contents list is created each time `__init__` runs.

A method

```
def put(self, item):
    if len(self.contents) < self.max_size:
        self.contents.append(item)
    else:
        print("No Room!")
```

This definition belongs inside Backpack.

The method reads the capacity and changes the contents of the receiving object.

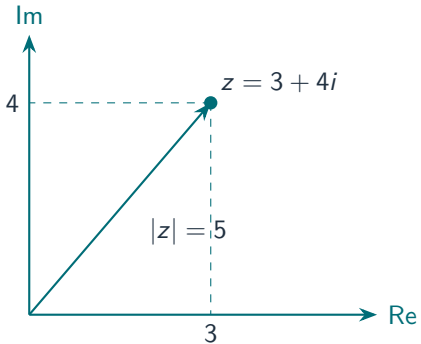
Instances

```
pack = Backpack("Barry", "black")
other = Backpack("Ada", "blue")
pack.put("book")
```



In `pack.put("book")`, Python supplies `pack` as self.
We pass only the item explicitly.

Complex numbers



State
real part: 3
imaginary part: 4

Behaviour
conjugate: $3 - 4i$
magnitude: 5
addition with another number

A mathematical object gives us a natural state and natural operations.

ComplexNumber

```
class ComplexNumber:
    def __init__(self, real, imag=0):
        self.real = real
        self.imag = imag

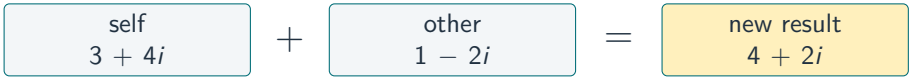
    def conjugate(self):
        return ComplexNumber(self.real, -self.imag)
```

The result is another object of the same class.

The original number keeps its real and imaginary parts.

Addition

```
def __add__(self, other):
    return ComplexNumber(self.real + other.real,
                         self.imag + other.imag)
```



Here, `other` is one ComplexNumber.

A method can also accept a list of objects as an argument;
its implementation then loops over the list to work with each object.

Special methods

```
def __abs__(self):
    return (self.real ** 2 + self.imag ** 2) ** 0.5

def __str__(self):
    sign = "+" if self.imag >= 0 else "-"
    return "({0}{1}{2}j)".format(
        self.real, sign, abs(self.imag))
```

- These methods belong inside `ComplexNumber`.
- `abs(z)` calls `__abs__`; `print(z)` uses the string from `__str__`.
- In `.format(...)`, `{0}`, `{1}`, `{2}` insert the three arguments in order.

Using the class

```
z = ComplexNumber(3, 4)
w = ComplexNumber(1, -2)
print(z)
print(z.conjugate())
print(z + w)
print(abs(z))
```

Output

```
(3+4j)
(3-4j)
(4+2j)
5.0
```

- Conjugation and addition return new objects; `z` and `w` keep their values.
- Addition involves two objects: `z` is `self`, and `w` is `other`.

Python already has complex numbers, such as 3+4j; this class illustrates how they work.

Recap

Arrays
shape, slice, reshape, stack

Comprehensions
transform, filter, combine

Objects
class, instance, state, method, self

Explore

Companion notebook

`02-arrays-loops-objects-try-it.ipynb`
Run the examples, change the inputs and predict the next result.

Readings

ACME Lab 1 (loops); Lab 3 (objects); Lab 4 (NumPy).

Examples draw on ACME Labs and Python Essentials. Attribution: Dr. J. Humpherys, Foundations-of-Applied-Mathematics/Labs, CC BY 3.0 US. New diagrams and teaching examples added for DM587.

Python: __main__; NumPy: indexing; reshaping.